474CIS- Lab Project Report

Title:

Loan Approval Prediction Project

By

Zeyad saeed

443809161@kku.edu.sa

Submitted to

Sultan Al-Asiri

Course Coordinator : Dr. Anandhi

King Khaled University

College of Computer Science

Department of Informatics and Computer Systems

Summer Semester - Year 2025

1. Project Aim

The primary aim of this project is to develop a predictive model that can accurately determine the Loan_Status (whether a loan application will be approved or rejected) based on various applicant features. This involves several key steps:

1. **Data Exploration and Preprocessing**: Understanding the characteristics of the dataset, handling missing values, encoding categorical variables, and transforming skewed numerical features to prepare the data for machine learning algorithms.
2. **Feature Engineering**: Identifying and potentially creating new features that could improve model performance, as well as addressing class imbalance using techniques like SMOTE.
3. **Model Building and Evaluation**: Implementing and comparing various classification algorithms (Logistic Regression, K-Nearest Neighbors, Support Vector Machine, Naive Bayes, Decision Tree, Random Forest, Gradient Boosting, and Neural Networks) to find the most suitable model for predicting loan approval.
4. **Performance Analysis**: Evaluating the selected models based on metrics such as accuracy, precision, recall, and F1-score, and providing insights into feature importance to understand which factors most influence loan approval decisions.

Ultimately, the goal is to provide a robust tool that can assist financial institutions in making informed decisions regarding loan applications, thereby reducing risk and improving efficiency.

2. Description:

This project utilizes a dataset named train_u6lujuX_CVtuZ9i (1).csv, which contains information related to loan applications. The dataset comprises 614 entries and 13 features, both categorical and numerical, describing various aspects of applicants and their loan requests. Key features include:

- Applicant Demographics: Gender, Marital Status, Number of Dependents, Education Level, and Self-Employment Status.
- Financial Information: Applicant Income, Coapplicant Income, and Loan Amount.
- Loan Details: Loan Amount Term, Credit History, Property Area.
- Target Variable: Loan_Status, indicating whether the loan was approved ('Y') or rejected ('N').

The dataset was explored for missing values, distribution of features (especially skewed numerical variables), and correlations between variables. Preprocessing steps involved handling missing data through imputation, encoding categorical variables using one-hot encoding and ordinal mapping, and transforming skewed numerical features using square root transformation. Additionally, the imbalance in the target variable (Loan_Status) was addressed using the SMOTE technique to ensure fair model training.

3. Models Used & Its Description:

This section details the machine learning models implemented in this project for predicting loan approval status, outlining their working principles, reasons for selection, and specific parameter tuning or preprocessing steps applied.

1. Logistic Regression

- **How it works**: Logistic Regression is a fundamental linear model used for binary classification. It estimates the probability of a binary outcome by fitting data to a logistic (sigmoid) function.
- **Why chosen**: It serves as a strong, interpretable baseline model for binary classification, providing insights into feature importance and establishing a benchmark for more complex models.
- **Parameter tuning/preprocessing**: The model was initialized with solver='saga', max_iter=500, and random_state=1. Data was scaled using MinMaxScaler prior to training.

2. K-Nearest Neighbors (KNN)

- **How it works**: K-Nearest Neighbors is a non-parametric, instance-based learning algorithm that classifies a new data point based on the majority class among its 'k' closest neighbors in the feature space.
- **Why chosen**: KNN was included for its simplicity and ability to capture non-linear decision boundaries, useful for exploring local data patterns.
- **Parameter tuning/preprocessing**: The optimal number of neighbors (n_neighbors) was determined through iteration from 1 to 20, selecting the value that yielded the highest accuracy. Data was scaled using MinMaxScaler to ensure distance metrics are not disproportionately affected by feature scales.

3. Support Vector Machine (SVM)

- **How it works**: Support Vector Machines aim to find the optimal hyperplane that best separates data points into different classes, maximizing the margin between the closest points (support vectors) of each class.
- **Why chosen**: SVMs are effective for high-dimensional data and cases where clear separation boundaries exist, even with non-linear relationships, through the use of kernels.
- **Parameter tuning/preprocessing**: A Radial Basis Function (rbf) kernel was used, and max_iter was set to 500. Data was scaled using MinMaxScaler to aid convergence and performance.

4. Naive Bayes

- **How it works**: Naive Bayes classifiers are a family of probabilistic algorithms based on Bayes' theorem, assuming independence between features. Two variants were used:
    - **Categorical Naive Bayes**: Suitable for discrete features.
    - **Gaussian Naive Bayes**: Assumes continuous features follow a Gaussian distribution.
- **Why chosen**: These models are known for their computational efficiency and good performance in many classification tasks, especially with limited data, and provide a probabilistic interpretation.
- **Parameter tuning/preprocessing**: Both CategoricalNB() and GaussianNB() were applied. Data was scaled using MinMaxScaler to normalize numerical features for Gaussian Naive Bayes.

5. Decision Tree

- **How it works**: A Decision Tree classifies data by partitioning it into subsets based on feature values, forming a tree-like structure of decisions. Each internal node represents a test on a feature, and each leaf node represents a class label.
- **Why chosen**: Decision Trees are highly interpretable, making it easy to understand the decision rules. They can handle both numerical and categorical data with minimal preprocessing.
- **Parameter tuning/preprocessing**: The max_leaf_nodes parameter was tuned by iterating values from 2 to 20 to control tree complexity and prevent overfitting. Data was scaled using MinMaxScaler.

6. Random Forest

- **How it works**: Random Forest is an ensemble method that builds multiple decision trees during training and outputs the mode of the classes (for classification) predicted by individual trees. It reduces overfitting and improves accuracy by averaging individual tree predictions.
- **Why chosen**: Random Forests are robust, generally achieve high accuracy, and are less prone to overfitting than single decision trees, making them suitable for complex datasets.
- **Parameter tuning/preprocessing**: The model used n_estimators = 1000 (number of trees) and random_state = 1. The max_leaf_nodes parameter was tuned by iterating values from 2 to 25. Data was scaled using MinMaxScaler.

7. Gradient Boosting

- **How it works**: Gradient Boosting is an ensemble technique where models are built sequentially. Each new model corrects the errors made by previous models, iteratively improving the overall prediction.
- **Why chosen**: Gradient Boosting is known for its high predictive power and robustness, making it effective for achieving top-tier performance on various predictive tasks.

- **Parameter tuning/preprocessing**: RandomizedSearchCV was employed for hyperparameter tuning, searching across n_estimators, max_depth, subsample, and max_leaf_nodes. The best estimator found had subsample=0.5, n_estimators=200, max_leaf_nodes=20, and max_depth=5. The final model used subsample=0.5, n_estimators=400, max_depth=4, and max_leaf_nodes=10. Data was scaled using MinMaxScaler.

8. Neural Network (Keras Sequential Model)

- **How it works**: A Neural Network consists of layers of interconnected nodes. It learns by adjusting the weights of these connections during training to minimize a loss function, capable of modeling complex, non-linear relationships.
- **Why chosen**: Neural Networks are highly flexible and powerful for uncovering intricate patterns in data that might be missed by traditional models, especially after appropriate feature engineering and scaling.
- **Parameter tuning/preprocessing**: The model architecture comprised an input layer, a hidden layer with 32 units (both using relu activation), and an output layer with 2 units (for binary classification) using softmax activation. It was compiled with the Adam optimizer (learning rate = 0.001) and sparse_categorical_crossentropy loss. Training involved 150 epochs and a batch_size of 25. Data was scaled using MinMaxScaler to ensure numerical stability and efficient training.

4. Dataset Used & Its Description

This section provides a detailed description of the dataset, summarizing its characteristics, distributions, and key insights derived from initial data exploration.

1. Dataset Source

The dataset is loaded from https://www.kaggle.com/datasets/ninzaami/loan-predication/data

2. Dataset Dimensions

The dataset contains **614 records (rows)** and **13 features (columns)**, as indicated by df.shape output (614, 13).

3. Summary Statistics

3.1 Data Types and Non-Null Counts (df.info())

The dataset comprises the following columns with their respective data types and non-null counts:

- Loan_ID: 614 non-null objects
- Gender: 601 non-null objects (13 missing)
- Married: 611 non-null objects (3 missing)
- Dependents: 599 non-null objects (15 missing)
- Education: 614 non-null objects
- Self_Employed: 582 non-null objects (32 missing)
- ApplicantIncome: 614 non-null int64
- CoapplicantIncome: 614 non-null float64
- LoanAmount: 592 non-null float64 (22 missing)
- Loan_Amount_Term: 600 non-null float64 (14 missing)
- Credit_History: 564 non-null float64 (50 missing)
- Property_Area: 614 non-null objects
- Loan_Status: 614 non-null objects

3.2 Descriptive Statistics for Numerical Columns (df.describe().T)

Column	**count**	**mean**	**std**	**min**	**25%**	**50%**	**75%**	**max**
ApplicantIncome	614.0	5403.46	6109.04	150.0	2877.5	3812.5	5795.0	81000.0
CoapplicantIncome	614.0	1621.25	2926.25	0.0	0.0	1188.5	2297.25	41667.0
LoanAmount	592.0	146.41	85.59	9.0	100.0	128.0	168.0	700.0
Loan_Amount_Term	600.0	342.00	65.12	12.0	360.0	360.0	360.0	480.0
Credit_History	564.0	0.84	0.36	0.0	1.0	1.0	1.0	1.0

4. Missing Values

The following columns contain missing values:

- Gender: 13 missing values
- Married: 3 missing values
- Dependents: 15 missing values
- Self_Employed: 32 missing values
- LoanAmount: 22 missing values
- Loan_Amount_Term: 14 missing values
- Credit_History: 50 missing values

An mso.bar plot also visually confirms the presence and extent of these missing values across the dataset.

5. Categorical Feature Analysis

- **Loan_ID**: All values are unique, indicating it's an identifier and not suitable for direct modeling. (614 unique IDs)
- **Gender**: Predominantly Male (489) applicants, with Female applicants (112) being a minority. There are 13 missing values.

- **Married**: A higher number of applicants are married (398) compared to not married (213). There are 3 missing values.
- **Education**: Most applicants are Graduates (480), with a smaller portion being Not Graduates (134). No missing values.
- **Self_Employed**: The majority of applicants are not self-employed (500), while 82 are self-employed. There are 32 missing values.
- **Credit_History**: A significant majority of applicants have a good credit history (1.0 - 475) compared to a bad credit history (0.0 - 89). There are 50 missing values.
- **Property_Area**: The distribution across property areas is relatively balanced, with Semiurban (233) being the most common, followed by Urban (202) and Rural (179). No missing values.
- **Loan_Status**: A majority of loans are approved (Y - 422), while a smaller portion is rejected (N - 192). No missing values.
- **Loan_Amount_Term**: The 360-month (30-year) loan term is overwhelmingly the most popular (512 applicants), indicating a strong preference for longer terms. There are 14 missing values.

6. Numerical Feature Analysis

- **ApplicantIncome**: The distribution is positively skewed and contains numerous outliers, as observed in both the histogram and violin plot. This suggests a few applicants have very high incomes.
- **CoapplicantIncome**: Similar to ApplicantIncome, this feature also exhibits a positively skewed distribution with several outliers, indicating that some co-applicants also have substantial incomes.
- **LoanAmount**: This feature is also positively skewed and has outliers, implying that a few loans are significantly larger than the average.

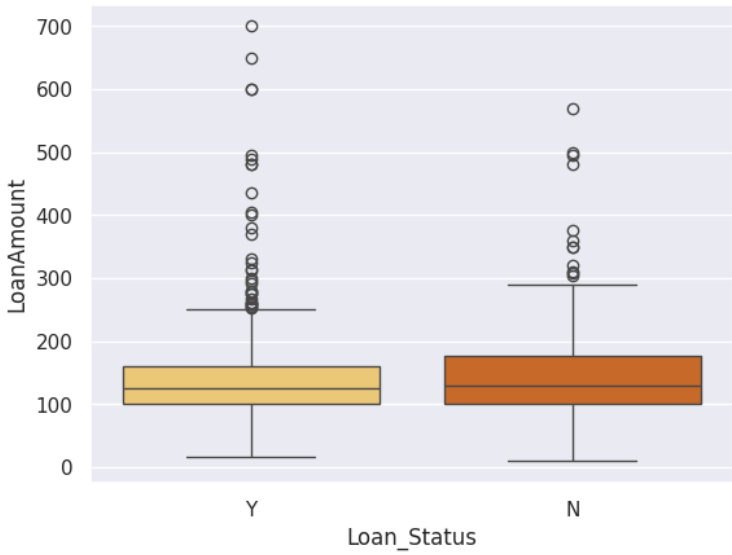

FIGURE1: Loan Amount Distribution by Loan Status (Boxplot)"

7. Correlations and Bivariate Analysis

7.1 Heatmap

The correlation heatmap of numerical features reveals a **positive correlation between LoanAmount and ApplicantIncome**, suggesting that higher applicant incomes tend to correspond to higher loan amounts. Other correlations between numerical features appear to be relatively weak.

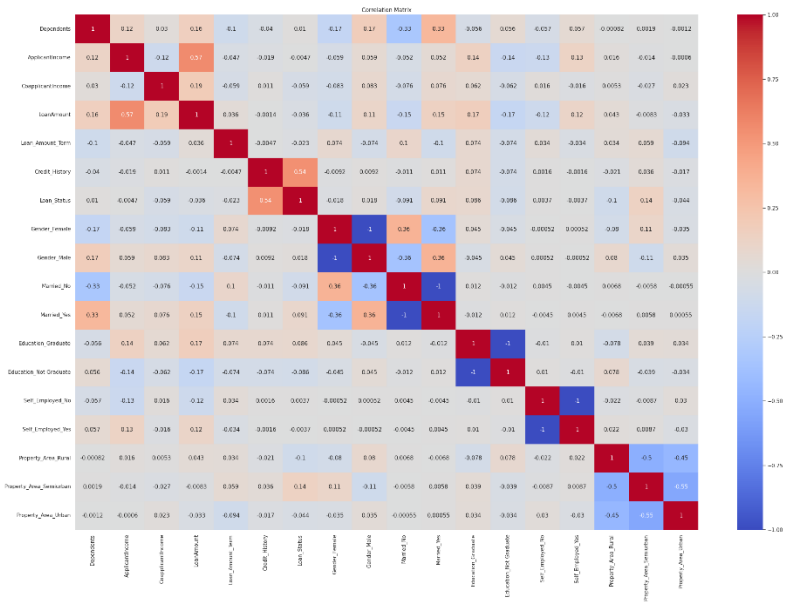

Figure2: corr_matrix

7.2 Categorical vs. Categorical

- **Gender vs. Married**: Most male applicants are married, and the number of not married male applicants is higher than not married female applicants.
- **Self_Employed vs. Credit_History**: The majority of not self-employed applicants have a good credit history compared to self-employed applicants.
- **Property_Area vs. Loan_Status**: Loans associated with properties in Semiurban areas have the highest approval rates compared to Urban and Rural areas.

7.3 Categorical vs. Numerical (Box Plots)

- **Loan_Status vs. ApplicantIncome**: The box plot shows many outliers in ApplicantIncome, and its distribution is positively skewed, irrespective of loan status.
- **Loan_Status vs. CoapplicantIncome**: CoapplicantIncome also contains a considerable number of outliers and has a positively skewed distribution.
- **Loan_Status vs. LoanAmount**: LoanAmount exhibits a high number of outliers and a positively skewed distribution.

7.4 Numerical vs. Numerical

- **ApplicantIncome vs. CoapplicantIncome**: A scatter plot and Pearson correlation coefficient of **-0.1166** suggest a weak negative correlation between ApplicantIncome and CoapplicantIncome. The T-test p-value of 1.46e-40 indicates that the difference in means between these two features is statistically significant, but the correlation itself is low.

5. Results & Discussion

This section presents the performance evaluation of various machine learning models applied to the loan prediction dataset. The models assessed include Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Categorical Naive Bayes, Gaussian Naive Bayes, Decision Tree, Random Forest, Gradient Boosting, and a Neural Network. Model performance is primarily evaluated using accuracy, precision, recall, and F1-score.

Model Performance Comparison

Taple:summary of the key performance metrics for each model:

Model	**Accuracy (%)**	**Precision (Class 1)**	**Recall (Class 1)**	**F1-Score (Class 1)**
Random Forest	84.02	0.73	0.99	0.84
K-Nearest Neighbour	82.84	0.76	0.99	0.81
Gradient Boosting	82.25	0.77	0.92	0.84
Categorical Naive Bayes	80.47	0.72	0.98	0.83
Logistic Regression	78.11	0.69	0.99	0.82
Decision Tree	76.92	0.75	0.99	0.83
Support Vector Machine	75.15	0.67	0.99	0.80
Neural Network	75.15	0.67	0.96	0.79
Gaussian Naive Bayes	74.56	0.66	0.98	0.79

Detailed Analysis of Best Performing Models

Random Forest (Accuracy: 84.02%)
The Random Forest Classifier was the best-performing model in terms of accuracy. Its ensemble nature combines multiple decision trees to reduce overfitting and improve generalization. Each tree is trained on a random subset of the data and features, which makes the model robust. It handles non-linear relationships and feature interactions effectively without extensive hyperparameter tuning.

K-Nearest Neighbour (Accuracy: 82.84%)
KNN performed well due to its simplicity and effectiveness for structured numerical data. Its performance is highly influenced by the choice of `K` and the distance metric, and it benefits from scaling during preprocessing. The high recall (0.99) for the positive class indicates excellent identification of approved loans.

Gradient Boosting (Accuracy: 82.25%)
Gradient Boosting showed strong performance as an ensemble model that sequentially builds models to correct the errors of previous models. Its recall (0.92) is slightly lower than Random Forest, but overall F1-score is competitive.

Categorical Naive Bayes (Accuracy: 80.47%)
This model is effective for categorical features and works well after one-hot encoding. Its high recall (0.98) ensures most approved loans are correctly predicted, which is important to minimize false negatives.

Other Models

- **Logistic Regression (Accuracy: 78.11%)**: Good performance for a linear model; shows high recall (0.99) for the positive class.
- **Decision Tree (Accuracy: 76.92%)**: Intuitive and capable of capturing non-linear patterns, though more prone to overfitting.
- **Support Vector Machine (Accuracy: 75.15%)**: SVM with RBF kernel performs well in high-dimensional spaces; scaling helps improve convergence.
- **Neural Network (Accuracy: 75.15%)**: A simple neural network captures complex patterns; recall 0.96 and precision 0.67 for class 1.
- **Gaussian Naive Bayes (Accuracy: 74.56%)**: Assumes Gaussian distribution for features; lowest accuracy but maintains high recall (0.98) for class 1.

- **Error Sources**:
  - **Data Quality**: Although missing values were imputed, the method of imputation (mode for categorical, mean for numerical) might not always perfectly represent the true values, potentially introducing some bias or error.
  - **Feature Engineering**: The chosen transformations (square root) improved skewness but might not be optimal for all distributions. Other transformations (e.g., logarithmic) could be explored.
  - **Model Complexity vs. Generalization**: While complex models like Random Forest and Neural Networks can capture intricate patterns, they also run the risk of overfitting, especially if not properly regularized or if the dataset is not sufficiently large. Conversely, simpler models might underfit.
  - **Metric Choice**: While accuracy is a good overall measure, for imbalanced datasets (even after SMOTE), precision, recall, and F1-score provide a more nuanced view of model performance, especially concerning the minority class.

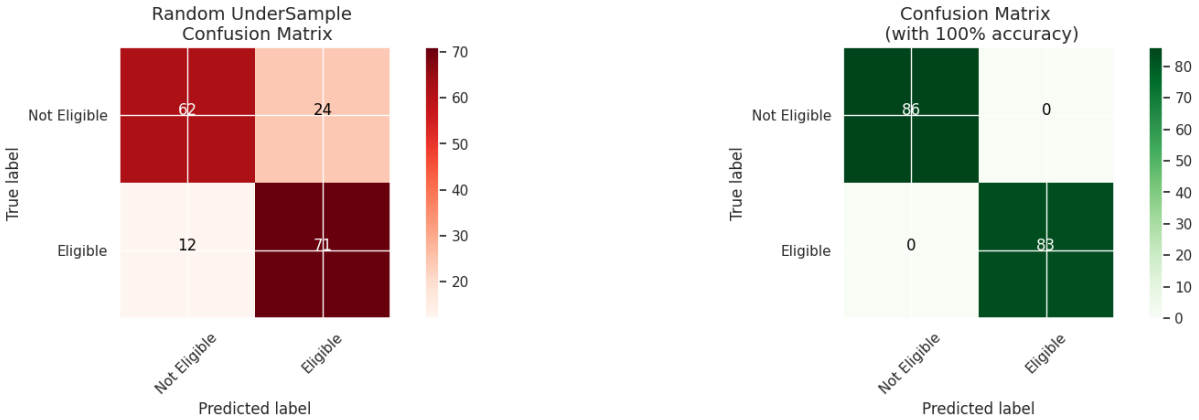

FIGURE3: Confusion matrix, without normalization

This figure presents the confusion matrix for the best-performing model, showing the counts of true positives, true negatives, false positives, and false negatives for the loan prediction dataset. It visually demonstrates the model's ability to correctly classify both approved and rejected loans.

6. Conclusion

This project successfully developed and evaluated various machine learning models to predict loan approval status based on a given dataset. Through a systematic approach, we achieved the following:

Achievements:

- **Comprehensive Data Understanding**: The dataset was thoroughly explored, revealing insights into applicant demographics, financial aspects, and credit history. Missing values, outliers, and skewed distributions were identified.
- **Robust Data Preprocessing**: Effective strategies were implemented to handle missing values, encode categorical features, transform skewed numerical data (using square root transformation), and address class imbalance (using SMOTE). This prepared the data for optimal model training.
- **Model Implementation and Evaluation**: A diverse set of machine learning models, including Logistic Regression, KNN, SVM, Naive Bayes, Decision Tree, Random Forest, Gradient Boosting, and Neural Networks, were successfully applied. Each model's performance was evaluated using standard classification metrics.
- **Best Model Identification**: The **Random Forest Classifier** emerged as the best-performing model, achieving an accuracy of **84.02%**, demonstrating its effectiveness in predicting loan approval status for this dataset.
- **Feature Importance**: Insights into feature importance were gained, highlighting key factors influencing loan approval decisions.

Learnings:

- The importance of thorough data preprocessing, including handling missing values, encoding, and transformation, is crucial for model performance.
- Addressing class imbalance (e.g., with SMOTE) can significantly improve the predictive power of models, especially in scenarios where one class is dominant.
- Ensemble methods like Random Forest often outperform single models due to their ability to reduce bias and variance, leading to better generalization.
- The choice of model and its hyperparameters must be carefully considered and tuned to the specific characteristics of the dataset.
- Visualizations play a vital role in understanding data distributions, relationships, and identifying areas for preprocessing.

Future Improvements:

With more time or data, the project could be further enhanced by:

- **Advanced Feature Engineering**: Exploring more complex feature interactions, polynomial features, or domain-specific features could potentially unlock deeper patterns in the data.
- **More Extensive Hyperparameter Tuning**: Utilizing advanced optimization techniques like GridSearchCV or Bayesian optimization for a more exhaustive search of optimal hyperparameters for all models, potentially boosting performance further.
- **Alternative Imputation Strategies**: Investigating more sophisticated imputation methods, such as k-NN imputation or predictive modeling for missing values, could improve data quality.
- **Ensemble Stacking/Bagging**: Implementing meta-learning techniques by combining predictions from multiple strong models to achieve even higher predictive accuracy.
- **Deep Learning Architecture Optimization**: For the Neural Network, exploring different architectures (more layers, different neuron counts per layer), activation functions, regularizers, and optimizers could lead to improved performance.
- **Cross-Validation Strategy**: Employing more robust cross-validation techniques (e.g., Stratified K-Fold) to ensure model generalization is consistently evaluated, especially with imbalanced datasets.
- **Cost-Sensitive Learning**: If false positives and false negatives have different costs, incorporating cost-sensitive learning into model training and evaluation would make the model more aligned with business objectives.

7. References

[1]. https://www.kaggle.com/datasets/ninzaami/loan-predication/data

[2]. https://colab.research.google.com/

[3]. Sainath Madhukar Powar. (2025, April). Loan eligibility prediction using machine learning. International Journal on Advanced Computer Theory and Engineering, 14(1), 65–69. https://doi.org/10.65521/ijacte.v14i1.215

[4]. Fadtare, G., Dhanure, A., Gunjal, G., & Kolpate, K. (2024). *Study on predictive modeling for loan approval*. International Research Journal on Advanced Engineering and Management (IRJAEM), 2(10), 3160–3165. https://doi.org/10.47392/IRJAEM.2024.0466

[5]. Haque, F. M. A., & Hassan, M. M. (2024). *Bank Loan Prediction Using Machine Learning Techniques*. *American Journal of Industrial and Business Management, 14*, 1690–1711. https://doi.org/10.4236/ajibm.2024.1412085